

Heaven's light is our guide"

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering

Compiler Design

Course No. : 701

Chapter 1: Introduction to compiling

Prepared By : Julia Rahman

1.1 Compilers

◆ What is a compiler?

- ✓ A compiler acts as a translator.
- ✓ A computer program that can read a program in one language - the source language - and translate it into an equivalent program in another language - the target language.

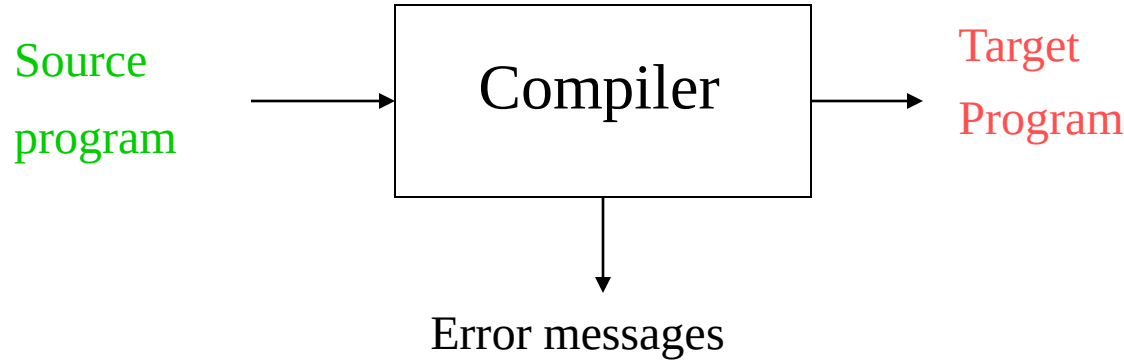


Figure 1.1: A compiler

- ✓ A **source program/code** is a program/code written in the source language, which is usually a high-level language.
- ✓ A **target program/code** is a program/code written in the target language, which often is a machine language or an intermediate code.
 - If the target program is an executable machine-language program, it can then be called by the user to process inputs and produce outputs; see Fig. 1.2

1.1 Compilers



Figure 1.2: Running the target program

Interpreter:

- ✓ Another common kind of language processor.
- ✓ Instead of producing a target program as a translation, an interpreter appears to directly execute the operations specified in the source program on inputs supplied by the user.



Figure 1.3: An interpreter

- ✓ The machine-language target program produced by a compiler is usually much faster than an interpreter at mapping inputs to outputs .
- ✓ An interpreter, however, can usually give better error diagnostics than a compiler, because it executes the source program statement by statement.

1.1 Compilers



Example 1.1 : Java language processors combine compilation and interpretation, as shown in Fig. 1.4. A Java source program may first be compiled into an intermediate form called bytecodes. The bytecodes are then interpreted by a virtual machine. A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network. In order to achieve faster processing of inputs to outputs, some Java compilers, called just-in-time compilers, translate the bytecodes into machine language immediately before they run the intermediate program to process the input.

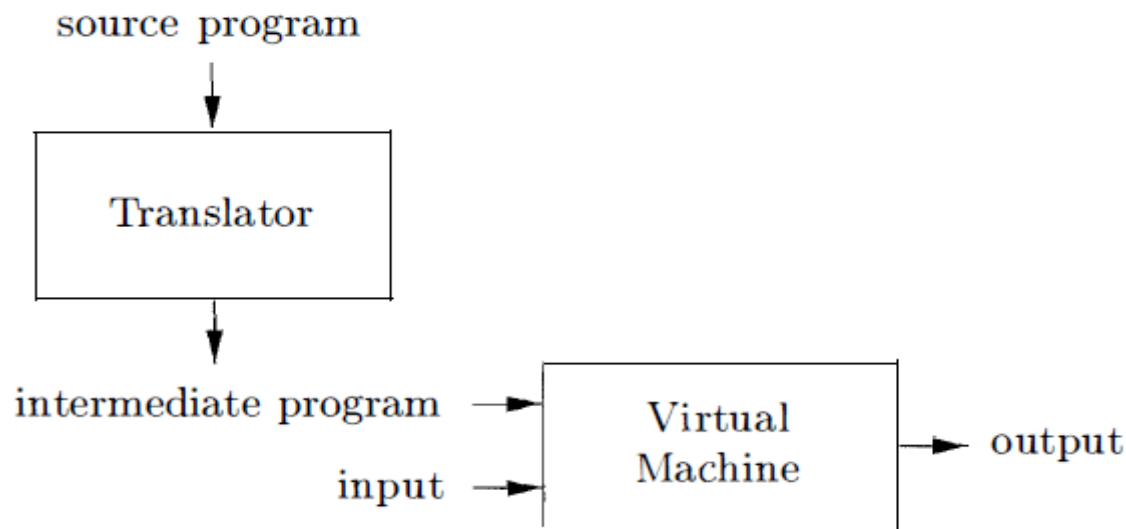


Figure 1.4: A hybrid compiler

1.1 Compilers

□ Compilers may generate three types of code:

▶ Pure Machine Code

- Machine instruction set without assuming the existence of any operating system or library.
- Mostly being OS or embedded applications.

▶ Augmented Machine Code

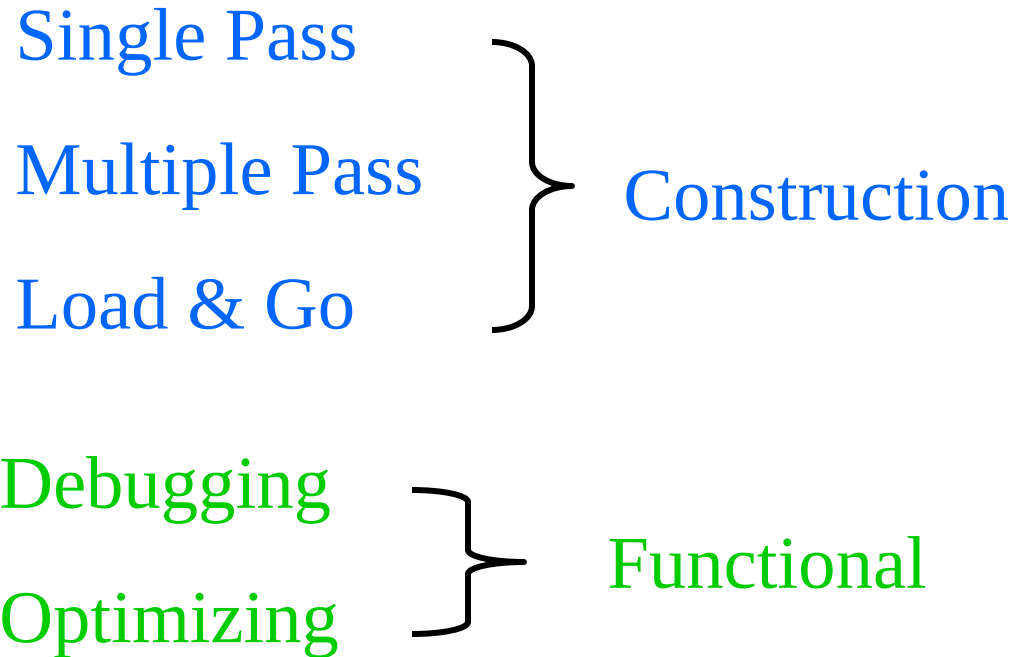
- Code with OS routines and runtime support routines.
- More often

▶ Virtual Machine Code

- Virtual instructions, can be run on any architecture with a virtual machine interpreter or a just-in-time compiler
- Ex. Java

1.1 Compilers

+ Classification of Compilers:



❖ However, All utilize same basic tasks to accomplish their actions

1.1 Compilers

The Analysis-Synthesis Model of Compilation:

The TWO Fundamental Parts:

Analysis: Decompose Source program into an intermediate representation

Synthesis: Target program generation from representation

The analysis part is often called the *front end* of the compiler; the synthesis part is the *back end*.

There are many **Software Tools** for helping with the Analysis Part:

➤ **Structure / Syntax directed editors:**

- ✓ Takes as input a sequence of commands to build a source program.
- ✓ Force “syntactically” correct code to be entered

➤ **Pretty Printers:**

- ✓ Standardized version for program structure (i.e., blank space, indenting)

➤ **Static Checkers:**

- ✓ A “quick” compilation to detect rudimentary errors

➤ **Interpreters:**

- ✓ “real” time execution of code a “line-at-a-time”

1.1 Compilers



Context of a compiler:

Skeletal source program



Preprocessor



Modified source program



Compiler



Target assembly program



Assembler



Relocatable machine code



Linker/Loader



Target machine code

library files
relocatable object files



Figure 1.5: A language-processing system

1.2 Analysis of the Source Program

The Analysis Task For Compilation:

Three Phases:

1) Linear / Lexical Analysis:

- ✓ Read Left -to-right and group into Tokens
- ✓ token: sequence of chars having a collective meaning

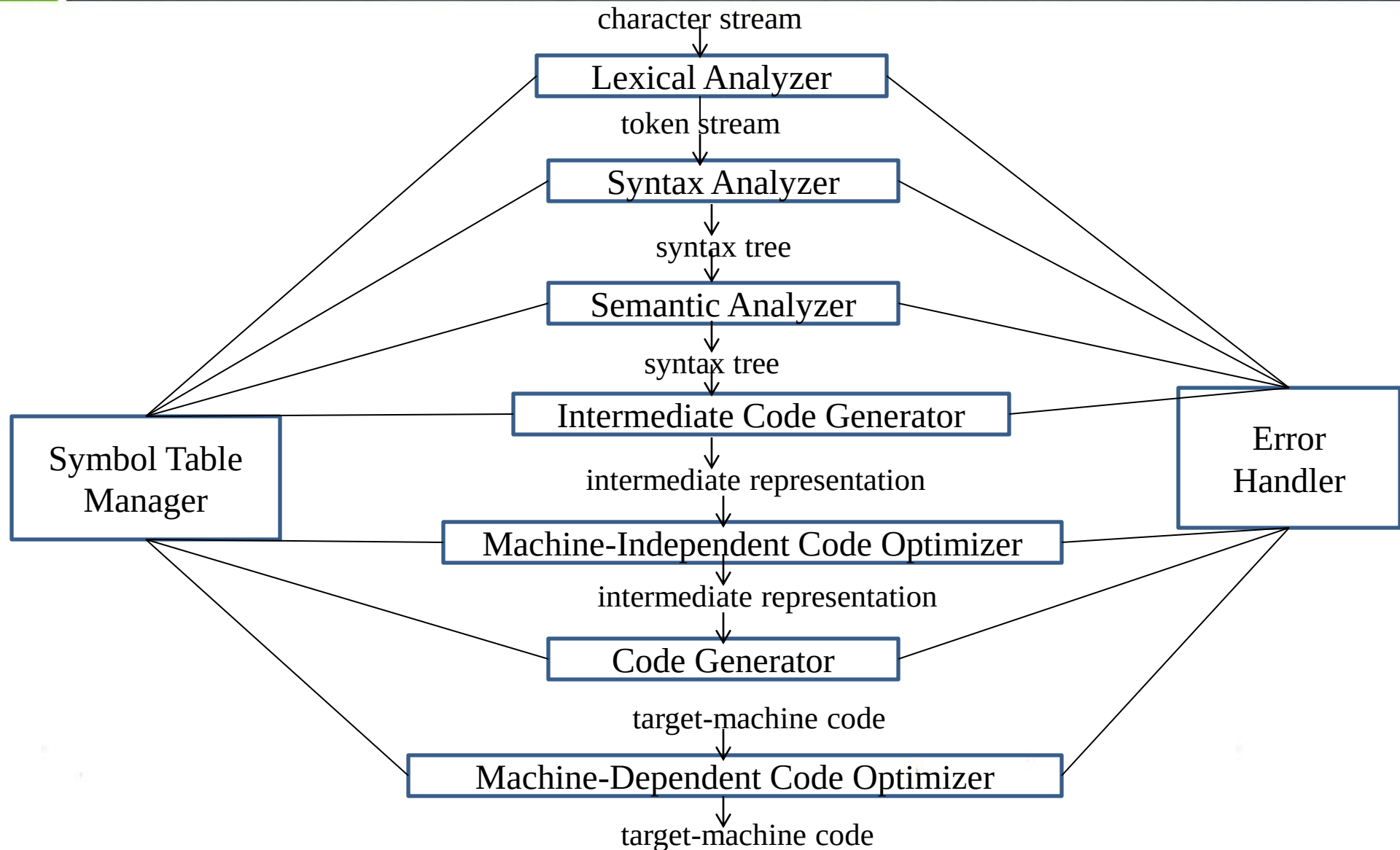
2) Hierarchical Analysis:

- ✓ Grouping of Tokens Into Meaningful Collection

3) Semantic Analysis:

- ✓ Checking to ensure Correctness of Components

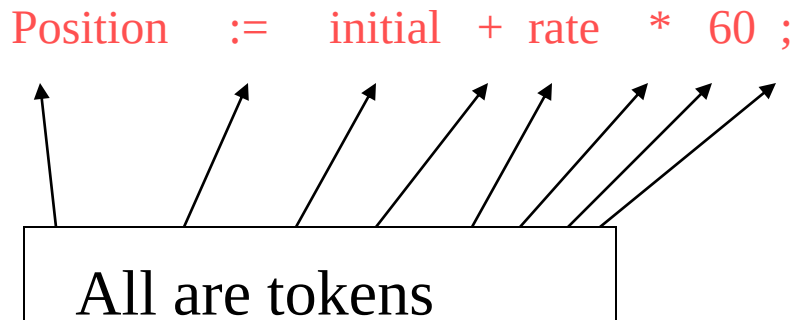
1.2 Analysis of the Source Program



1.3 The Phase of a Compiler

Lexical Analysis:

- ✓ The first phase of a compiler is called lexical analysis or scanning.
- ✓ The lexical analyzer reads the stream of characters making up the source program and groups the characters into meaningful sequences called lexemes.
- ✓ Identify tokens which are the basic building blocks.
- ✓ For example

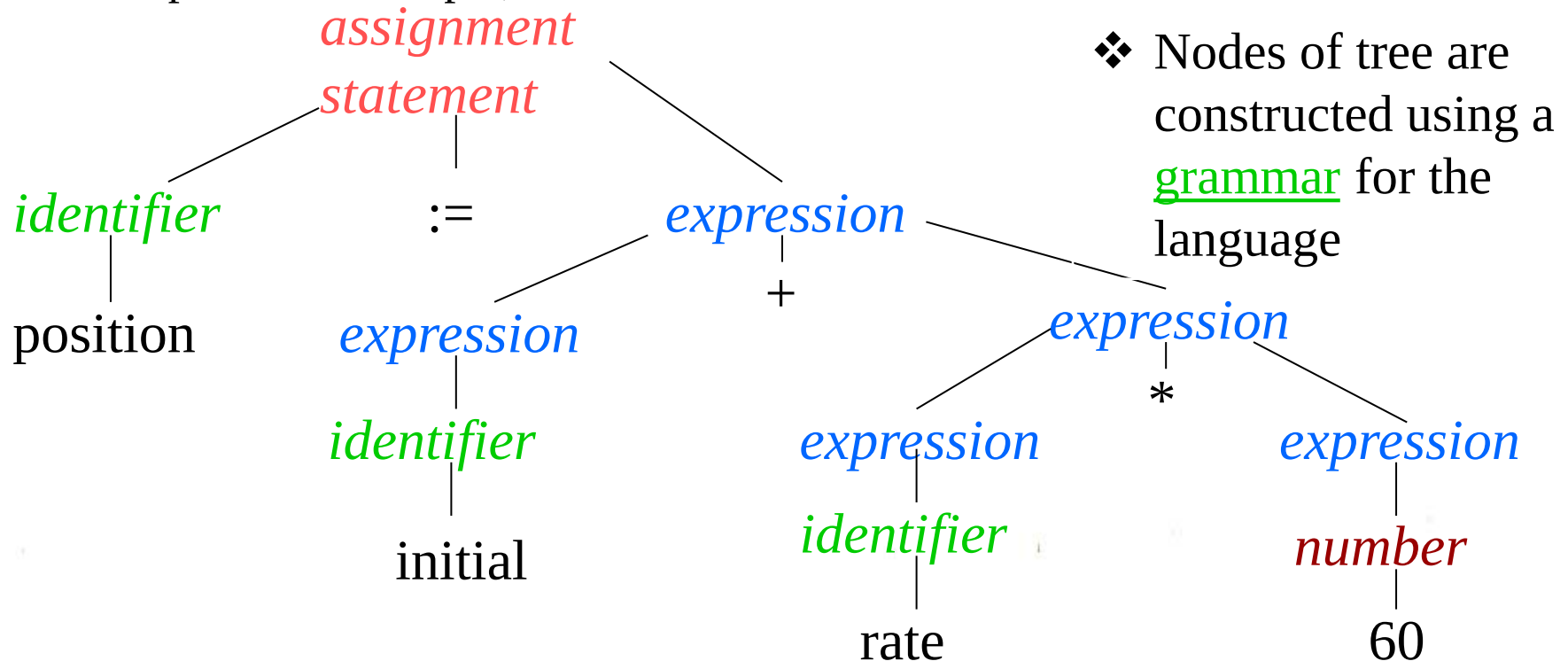


- ✓ Blanks, Line breaks, etc. are scanned out

1.3 The Phase of a Compiler

Syntax Analysis:

- ✓ The second phase of the compiler is syntax analysis or parsing.
- ✓ Involves grouping the tokens of the source program into grammatical phrases that are used by the compiler to synthesize output.
- ✓ In syntax tree each interior node represents an operation and the children of the node represent the arguments of the operation.
- ✓ For previous example, we would have Parse Tree:



1.3 The Phase of a Compiler

❑ *What is a Grammar?*

Grammar is a Set of Rules Which Govern the Interdependencies & Structure Among the Tokens

Statement $\Rightarrow$ is an assignment statement, or while statement, or if statement, or ..

Assignment statement $\Rightarrow$ is an identifier := expression ;

Expression $\Rightarrow$ is an (expression), or expression + expression, or expression * expression, or number, or identifier, or ...

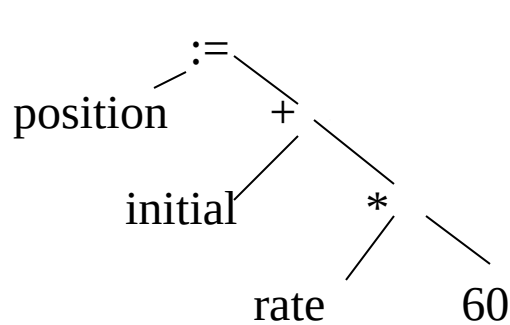
❑ *Why Have We Divided Analysis in This Manner?*

- *Lexical Analysis* - Scans Input, Its Linear Actions Are Not Recursive
 - Identify Only Individual “words” that are the Tokens of the Language
- Recursion is Required to Identify Structure of an Expression, As Indicated in Parse Tree
 - Verify that the “words” are Correctly Assembled into “sentences”
- *What is Third Phase?*
 - Determine Whether the Sentences have One and Only One Unambiguous Interpretation
 - ... and do something about it!
 - e.g. “John Took Picture of Mary Out on the Patio”

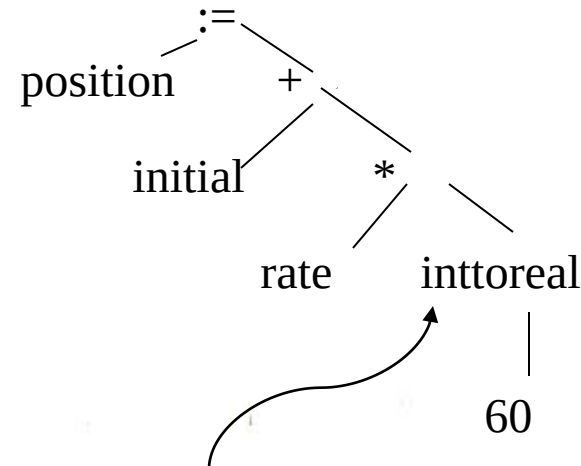
1.3 The Phase of a Compiler

Semantic Analysis:

- ✓ Checks the source program for semantic errors and gathers type information.
- ✓ Saves the information in either the syntax tree or the symbol table, for subsequent use during intermediate-code generation.
- ✓ Uses the syntax tree with Semantic Actions.
- ✓ An important part of semantic analysis is type checking, where the compiler checks that each operator has matching operands.
- ✓ For example, many programming language definitions require an array index to be an integer; the compiler must report an error if a floating-point number is used to index an array.



Compressed Tree



Conversion Action

1.3 The Phase of a Compiler

Symbol Table Creation / Maintenance:

- ✓ The ***symbol table*** is a data structure containing a record for each variable name, with fields for the attributes of the name.
- ✓ Contains Info (storage, type, scope, args) on Each “Meaningful” Token, Typically Identifiers
- ✓ Data Structure Created / Initialized During Lexical Analysis
- ✓ Utilized / Updated During Later Analysis & Synthesis
- ✓ The data structure should be designed to allow the compiler to find the record for each name quickly and to store or retrieve data from that record quickly.

Error Handling:

- ✓ Detection of different errors which correspond to all phases
- ✓ What Kinds of Errors Are Found During the Analysis Phase?
- ✓ What Happens When an Error Is Found?

1.3 The Phase of a Compiler

Intermediate Code Generation:

- ✓ After syntax and semantic analysis of the source program, many compilers generate an explicit low-level or machine-like intermediate representation of source program.
- ✓ This intermediate representation should have two important properties:
 - 1) It should be easy to produce and
 - 2) It should be easy to translate into the target machine.

Code Optimization:

- ✓ Attempts to improve the intermediate code so that better target code will result.
- ✓ Find More Efficient Ways to Execute Code
- ✓ Replace Code With More Optimal Statements
- ✓ 2-approaches: High-level Language & “Peephole” Optimization

Final Code Generation:

- ✓ Takes as input an intermediate representation of the source program and maps it into the target language.
- ✓ Generate Relocatable Machine Dependent Code

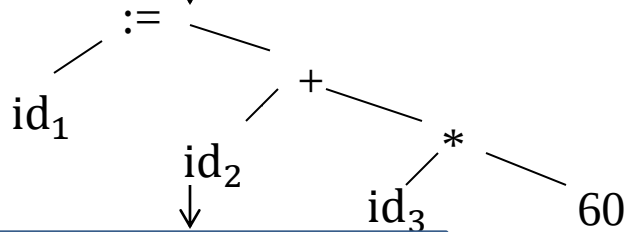
1.3 The Phase of a Compiler

position := initial + rate * 60

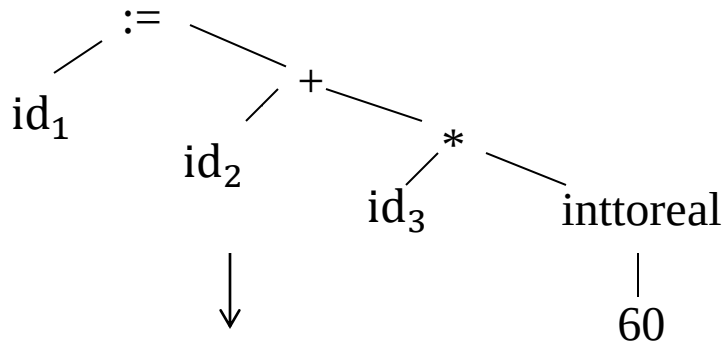
Lexical Analyzer

id₁ := id₂ + id₃ * 60

Syntax Analyzer



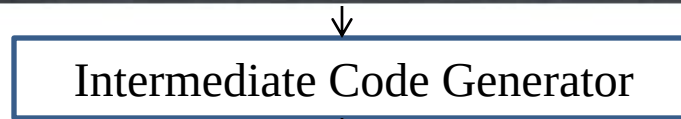
Semantic Analyzer



Symbol table

Position	
initial	
rate	

1.3 The Phase of a Compiler



↓

```
temp1 := inttoreal(60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1 := temp3
```



↓

```
temp1 := id3 * 60.0
id1 := id2 + temp1
```



```
MOVF id3 , R2
MULF #60.0, R2
MOVF id2 , R1
ADDF R1, R2
MOVF R1, id1
```

1.4 Cousins of the Compiler



Preprocessors:

- ✓ The task of collecting the source program is sometimes entrusted to a separate program, called a preprocessor.
- ✓ Produce input to compiler.
- ✓ Perform following functions:
 - 1) **Macro Processing:** A preprocessor may allow a user to define macros that are shorthands for longer constructs.
 - 2) **File inclusion:** Include Header file into a program. In C use `#include<stdio.h>` header.
 - 3) **“Rational” Preprocessors:** Augment older languages with more modern flow of control and data structure facilities.
 - 4) **Language Extensions:** Attempts to add capabilities to the language by what amounts to built in macros.

1.4 Cousins of the Compiler



Assemblers:

- ✓ Produces relocatable machine code as its output.
- ✓ The assembly language is processed by it.
- ✓ **Assembly code:** names are used for instructions, and names are used for memory addresses.

MOV a, R1

ADD #2, R1

MOV R1, b

- ✓ Two-pass Assembly:
 - **First Pass:** all identifiers are assigned to memory addresses and stored in a symbol table.

Example:

Identifiers

Address

a

0

b

4

- **Second Pass:** Translates each operation code into the sequence of bits representation. Produce relocatable machine code:

0001 01 00 00000000 * relocation

0011 01 10 00000010 bit

0010 01 00 00000100 *

1.4 Cousins of the Compiler

Loaders and Link-Editors



Loader:

Taking relocatable machine code, altering the addresses and placing the altered instructions into memory.



Link-editor:

Taking many (relocatable) machine code programs (with cross-references) and produce a single file.

- Need to keep track of correspondence between variable names and corresponding addresses in each piece of code.